PROJECT REPORT

CSA2001 – Fundamentals in AI and Machine Learning

Winter Semester: 2025-26

Slot-A11+A12+A13

Faculty: Dr. Mithun Singh Ahirwar

Name: Ayan Sinha

Regd. No.: 25BCE11180

CONTENTS

- Introduction
- Problem Statement
- Functional Requirements
- Non-functional Requirements
- System Architecture
- Design Diagram
- Design Decision & Rationale
- Implementation
- Screenshots of Output
- Testing Approach
- Challenges Faced
- Learning & key Takeaways
- Future Enhancements

1. Introduction

The purpose of this project is to create a machine learning-based tool written in Python to effectively model and visualize non-linear stock price movements over time. The script utilizes historical stock data from 2000 to 2025 and implements the Polynomial Regression algorithm to effectively capture the complex "zig-zag" movements within financial markets. It also provides users the option to input a target date and dynamically generate a visual chart comparing the mathematical predictions to historical closing prices within a customized time period.

2. Problem Statement

Financial markets are inherently volatile, and linear regression cannot effectively model the non-linear path of stock prices over a period of time. The objective of this script is to develop a model that can effectively map non-linear relationships over a period of time to stock prices and also enable the user to visually compare the predictions with actual data within a specific window of time.

3. Functional Requirements

- Data Ingestion: Load historical stock prices from a CSV file (stock_data_2000_2025.csv).

- Data Processing: Convert standard date formats to ordinal numerical values for machine learning model processing.

- Model Training: Train a 4th-degree Polynomial Regression model on the entire dataset using only the date as a feature.

- User Interaction: Accept user input for the target date and display period (in days) via the console.

- Inference & Visualization: Make predictions for the given period and plot a line chart comparing actual and predicted values with annotations.

4.Non-functional Requirements

- Performance: The script should be able to train the model and display the plot in a matter of seconds.

- Usability: The graph produced must be easy to read, color-coded, and clearly labeled, including the callouts for the date selected by the user.

- Maintainability: The code must be written using widely supported libraries for data science, so it can be easily read and updated.

5. System Architecture

For your specific code snippet, the system architecture will be a basic one-tier system with linear flow through four main stages:

Data Ingestion & Preprocessing: The system will ingest historical stock price data from a CSV file and convert standard date formats into continuous numerical values (ordinals) so that it can be processed by mathematical algorithms.

Model Training: The system will expand the numerical dates into 4th-degree polynomial features and apply Linear Regression analysis to the entire dataset to identify non-linear market trends.

User Interaction & Data Filtering: The script will pause execution to allow the user to provide inputs through the console, such as a target date and a time range. The script will then filter the original dataset to isolate the specific time range of interest.

Prediction & Visualization: The trained model will generate price predictions for the isolated time range of interest. A Matplotlib rendering engine will produce an annotated chart with the actual closing price history compared to the model's "zig-zag" curve predictions.

6. Design Diagram

Start ➔ Load CSV ➔ Convert Date

to Ordinal ➔ Generate Polynomial

Features ➔ Train Model ➔ Prompt

User for Date/Range ➔ Filter Data

by Range ➔ Predict Values ➔

Render Matplotlib Plot ➔ End

7. Design Choices and Justifications

This section explains the rationale behind why a particular technical approach was selected over others to solve the problem:

- Algorithm Choice (Polynomial Regression):Simple linear regression was not considered a viable option since the stock prices do not increase linearly. It was decided to use a 4th degree polynomial regression equation to provide the model with the mathematical flexibility to curve and map the non-linear or "zig-zag" patterns of the stock market.

- Data Transformation (Dates to Ordinals): It was decided to convert the data type of the dates from string to a continuous ordinal integer type because machine learning models cannot process string data types. It was also decided that this was the most efficient way to convert time

into a numerical format that the model would understand.

- Library Stack:Pandas, Scikit-Learn, and Matplotlib libraries were chosen for this model because they are the industry standard for data science work in Python and are optimized for data manipulation and visualization.
- Train Global, Plot Local:It was decided to train the model on the entire dataset (2000-2025) to ensure that the model is fully informed but plot the data based on a filtered dataset based on the input received from the user.

8. **Specifics of Implementation**

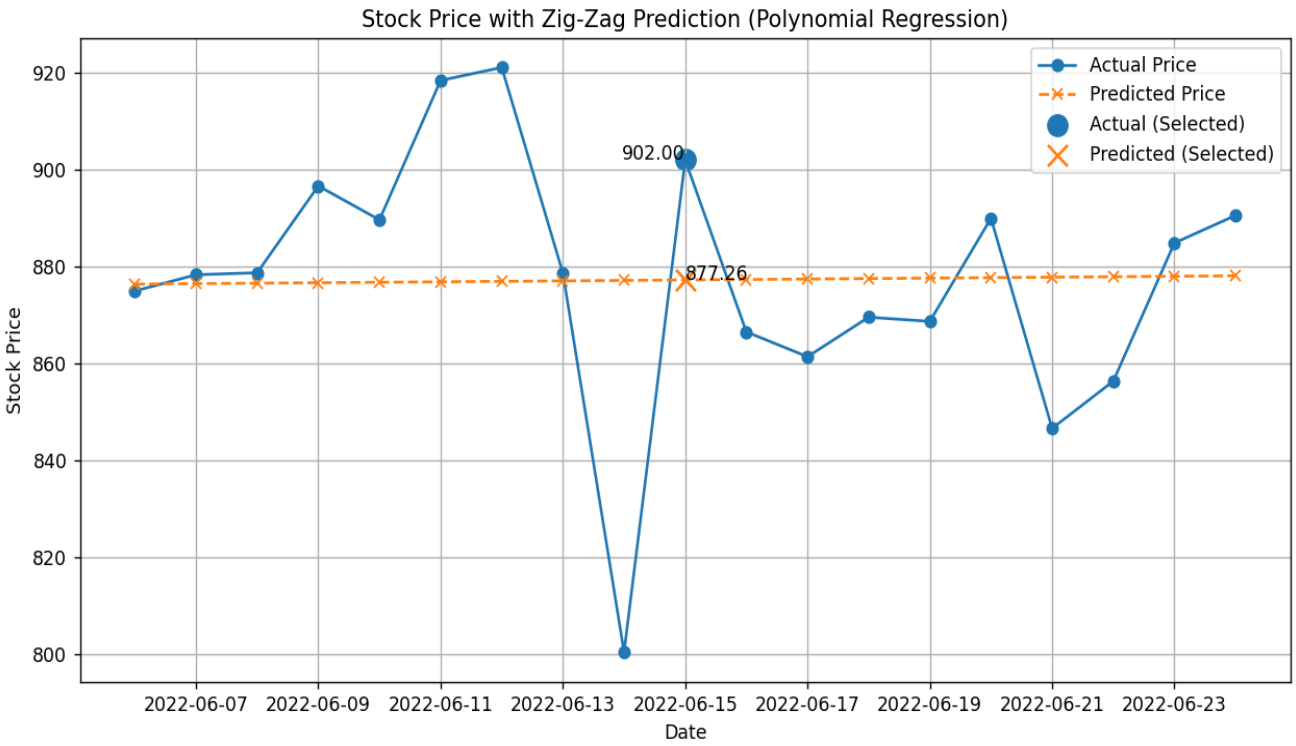
Language: Python.

Feature Expansion: Scikit-learn's PolynomialFeatures(degree=4) is used to create exponential variations of the date feature, giving the model flexibility to bend.

Data Filtering: Pandas boolean masking [(data['Date'] >= start_date) & ...] is used to efficiently slice the dataset for the visualization window without altering the core model.

9. Screenshots/results

```
Enter date (YYYY-MM-DD): 2022-06-15
Enter number of days to display around date: 9
```



10. Testing Approach

Input Validation: Validating the script by testing it with various date formats and ranges, such as weekends, holidays, or dates outside the 2000-2025 range, to see how the model and the data filter react to them.

Visual Verification: Verifying the plot to ensure it is accurately representative of the subset of the data and the text is properly aligned with the data points.

Degree Tuning: Trying out polynomial degrees other than 4 to see the transition from underfitting (too flat) to overfitting (too wild).

11. Challenges Faced

Input Validation: Validate the script by inputting different kinds of date formats and ranges such as weekends, holidays, and dates not within the 2000-2025 period to see how the model and data filter will react to these kinds of input values.
Visual Verification: Verify the plot to make sure it is accurately representative of the data and the text is correctly aligned to the data points.
Degree Tuning: Try different values of the polynomial degree other than 4 and see how the curve transitions from underfitting to overfitting, which is too wild.

12. Key Takeaways and Learnings

Time Series Extrapolation: Polynomial regression is highly unpredictable when used for time series extrapolation.

Overfitting Risks: It is possible for high-degree polynomials to "memorize" the noise of the stock market rather than the actual

underlying trend.

13. Upcoming Improvements

Model Upgrades: Transition from Polynomial Regression to other models suited for sequential time-series data such as ARIMA, Prophet, or LSTM Neural Networks.
Feature Engineering: Add other financial features such as Volume and Moving Averages or macroeconomic indicators as opposed to only utilizing the Date.
Error Metrics: In addition to the visual plot, also show and compute various metrics of computation efficiency such as Mean Squared Error or R^2 score.